

“ Introducción a los Sistemas Distribuidos”

“ Redes y Aplicaciones Distribuidas”

<75.43> <95.60> <TA048>

DATOS DEL TRABAJO PRÁCTICO

1	2026	
	AÑO	File Transfer
	1°	
TP NRO	CUAT	TEMA

INTEGRANTES DEL GRUPO

22			
	Fernandez Paz, Alejo	112142	
GRUPO	APELLIDO Y NOMBRE	PADRÓN	

Índice

Introducción.....	3
Hipótesis y suposiciones realizadas.....	3
1.1. Desarrollo.....	3
2. Implementación.....	4
2.1. Server-client.....	4
3. Pruebas.....	4
3.1. Configuración del entorno.....	4
3.2. Archivos Utilizados.....	5
3.3. Metodología.....	5
3.4. Resultados obtenidos.....	5
3.5. Observaciones.....	6
4. Preguntas a responder.....	7
4.1. Describa la arquitectura Cliente-Servidor.....	7
4.2. ¿Cuál es la función de un protocolo de capa de aplicación?.....	7
4.3. Detalle el protocolo de aplicación desarrollado en este trabajo.....	7
4.4. La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno?.....	7
4.5. Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión, en caso de tenerlos, descríbalos.....	7
4.6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?.....	7
5. Dificultades encontradas.....	7
Cierre prematuro del receptor.....	7
Pérdida del último ACK.....	8
Manejo de retransmisiones en el handshake.....	8
Pruebas en entorno simulado.....	8
6. Conclusión.....	8

Introducción

El presente trabajo práctico tiene como objetivo el desarrollo de una aplicación de transferencia de archivos sobre el protocolo UDP, implementando mecanismos de confiabilidad a nivel de aplicación. Dado que UDP no garantiza la entrega de paquetes ni el orden de los mismos, se implementaron dos protocolos de recuperación de errores: Stop & Wait y Go-Back-N.

La aplicación permite realizar operaciones de subida y descarga de archivos entre un cliente y un servidor, utilizando sockets UDP. Para garantizar la correcta transferencia de datos, se incorporaron mecanismos de control de secuencia, confirmaciones (ACKs), retransmisiones ante pérdidas y control de timeouts.

Además, se realizaron pruebas en un entorno simulado mediante Mininet, evaluando el comportamiento de ambos protocolos bajo diferentes condiciones de red, incluyendo escenarios con pérdida de paquetes.

Hipótesis y suposiciones realizadas

1.1. Desarrollo

Para el desarrollo de la aplicación se asumió que:

- La comunicación se realiza sobre UDP, por lo que no existe garantía de entrega, orden ni duplicación de paquetes.
- Los paquetes pueden perderse, duplicarse o llegar fuera de orden.
- El tamaño máximo de los paquetes está limitado por una constante predefinida (MAX_PACKET_SIZE).
- El cliente y el servidor comparten el mismo formato de paquetes y protocolo de comunicación.
- Se utiliza un mecanismo de numeración de secuencia para identificar cada paquete de datos.
- Se emplean timeouts para detectar la pérdida de paquetes o ACKs.
- En Go-Back-N, el receptor solo acepta paquetes en orden y descarta aquellos fuera de secuencia.
- La red puede presentar pérdida de paquetes (simulada con Mininet), lo que obliga a retransmisiones.

Estas suposiciones permiten modelar un entorno realista donde es necesario implementar confiabilidad sobre un protocolo no confiable como UDP.

2. Implementación

2.1. Server-client

La aplicación está compuesta por un cliente y un servidor que se comunican mediante sockets UDP. El servidor se encarga de atender múltiples solicitudes de clientes mediante el uso de threads, permitiendo la concurrencia en las transferencias.

Para cada operación de transferencia (upload o download), se realiza un handshake inicial en el que el cliente solicita la operación y el servidor responde con un puerto dinámico donde se llevará a cabo la transferencia de datos.

Se implementaron dos protocolos de recuperación de errores:

- Stop & Wait: el emisor envía un paquete y espera su confirmación antes de enviar el siguiente. En caso de timeout, el paquete es retransmitido.
- Go-Back-N: el emisor puede enviar múltiples paquetes sin esperar confirmación, manteniendo una ventana de transmisión. El receptor envía ACKs acumulativos y descarta paquetes fuera de orden. Ante la pérdida de un paquete, el emisor retransmite todos los paquetes a partir del último no confirmado.

Para garantizar la robustez del sistema, se implementaron mecanismos adicionales como:

- Reenvío del último ACK en caso de recibir paquetes duplicados.
- Manejo de pérdida del ACK final mediante un período de espera adicional (linger).
- Control de sesiones activas en el servidor para evitar la creación de múltiples transferencias ante retransmisiones del cliente.

Estos mecanismos permiten asegurar la correcta transferencia de archivos incluso en presencia de pérdida de paquetes.

3. Pruebas

Para evaluar el correcto funcionamiento de la aplicación, se realizaron distintas pruebas utilizando el emulador de redes Mininet. Este entorno permitió simular condiciones de red controladas, incluyendo pérdida de paquetes.

Se llevaron a cabo pruebas de transferencia de archivos entre dos hosts (h1 como servidor y h2 como cliente), utilizando ambos protocolos implementados: Stop & Wait y Go-Back-N.

3.1. Configuración del entorno

Se utilizó una topología simple con dos hosts conectados a un switch:

```
sudo mn --topo single,2 --link tc,loss=X
```

donde X representa el porcentaje de pérdida de paquetes (0% y 10%).

El servidor se ejecutó en h1 y el cliente en h2, utilizando direcciones IP del rango 10.0.0.0/8.

3.2. Archivos Utilizados

Se realizaron pruebas con archivos de distintos tamaños para analizar el comportamiento de los protocolos:

- 100 KB
- 1 MB
- 5 MB

3.3. Metodología

Para cada combinación de protocolo, tamaño de archivo y porcentaje de pérdida, se midió el tiempo total de transferencia utilizando el comando `time`. A partir de estos valores, se calculó el throughput en Mbps.

Las pruebas se realizaron para:

- Stop & Wait (S&W)
- Go-Back-N (GBN)

bajo dos condiciones:

- Sin pérdida (0%)
- Con pérdida (10%)

3.4. Resultados obtenidos

Protocolo	Tamaño	Pérdida	Tiempo (s)	Throughput (Mbps)
GBN	100 KB	0%	0.037	22.1
S&W	100 KB	0%	0.039	21.0
GBN	1 MB	0%	0.049	171.2
S&W	1 MB	0%	0.063	133.2

GBN	5 MB	0%	0.091	460.7
S&W	5 MB	0%	0.196	214.0
GBN	100 KB	10%	12.087	0.068
S&W	100 KB	10%	17.592	0.047
GBN	1 MB	10%	102.381	0.082
S&W	1 MB	10%	248.094	0.034

3.5. Observaciones

En condiciones sin pérdida, ambos protocolos presentan tiempos similares para archivos pequeños, pero Go-Back-N mejora significativamente el rendimiento en archivos de mayor tamaño.

En presencia de pérdida (10%), se observa un aumento considerable en los tiempos de transferencia para ambos protocolos.

Stop & Wait se ve más afectado por la pérdida de paquetes, ya que cada retransmisión detiene completamente el envío de nuevos datos.

Go-Back-N, al permitir múltiples paquetes en tránsito, logra mantener un mejor desempeño incluso en escenarios adversos.

En todos los casos, se verificó que los archivos transferidos llegan correctamente al destino, confirmando la confiabilidad de la implementación.

4. Preguntas a responder

4.1. Describa la arquitectura Cliente-Servidor.

4.2. ¿Cuál es la función de un protocolo de capa de aplicación?

4.3. Detalle el protocolo de aplicación desarrollado en este trabajo.

4.4. La capa de transporte del stack TCP/IP ofrece dos protocolos: TCP y UDP. ¿Qué servicios proveen dichos protocolos? ¿Cuáles son sus características? ¿Cuándo es apropiado utilizar cada uno?

4.5. Justifique si el protocolo desarrollado cuenta con mecanismos de control de congestión, en caso de tenerlos, descríbalos.

4.6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?

5. Dificultades encontradas

Durante el desarrollo del trabajo se presentaron diversas dificultades relacionadas con la implementación de confiabilidad sobre UDP y el manejo de condiciones adversas de red.

Cierre prematuro del receptor

Inicialmente, el receptor finalizaba la transferencia luego de recibir el primer paquete, debido a un error en la condición de salida del ciclo principal. Esto provocaba que el resto de los paquetes fueran ignorados, generando fallas en la transferencia de archivos de mayor tamaño.

Este problema fue resuelto corrigiendo la condición de finalización, asegurando que el receptor continúe recibiendo datos hasta completar el tamaño total esperado del archivo.

Pérdida del último ACK

En algunos casos, la transferencia fallaba al finalizar debido a la pérdida del último ACK enviado por el receptor. Esto provocaba que el emisor continuara retransmitiendo el último paquete, mientras que el receptor ya había finalizado la transferencia.

Para solucionar este problema, se implementó un período adicional de espera (linger) en el receptor, durante el cual se reenvía el último ACK en caso de recibir paquetes duplicados.

Manejo de retransmisiones en el handshake

Bajo condiciones de pérdida, el cliente podía reenviar la solicitud inicial de transferencia (UPLOAD_REQ) al no recibir respuesta del servidor. Esto provocaba que el servidor interpretara cada solicitud como una nueva conexión, generando múltiples transferencias simultáneas del mismo archivo.

Este problema se resolvió incorporando un mecanismo de deduplicación en el servidor, que permite identificar solicitudes repetidas y reutilizar la misma sesión de transferencia en lugar de crear nuevas.

Pruebas en entorno simulado

La utilización de Mininet presentó desafíos iniciales relacionados con la configuración de rutas y la ejecución de comandos en los hosts virtuales. En particular, fue necesario ajustar las rutas de acceso a los archivos y asegurar la correcta ejecución del servidor en segundo plano.

Una vez resueltos estos inconvenientes, Mininet permitió validar correctamente el comportamiento del sistema bajo condiciones de pérdida controlada.

6. Conclusión

Se logró implementar una aplicación de transferencia de archivos confiable sobre UDP, incorporando mecanismos de control de errores mediante los protocolos Stop & Wait y Go-Back-N.

A través de las pruebas realizadas, se pudo comprobar que ambos protocolos permiten garantizar la correcta entrega de los datos incluso en presencia de pérdida de paquetes. Sin embargo, se observaron diferencias significativas en su rendimiento.

En particular, Go-Back-N demostró ser más eficiente que Stop & Wait, especialmente en transferencias de mayor tamaño y en condiciones de red adversas. Esto se debe a su capacidad de mantener múltiples paquetes en tránsito simultáneamente, reduciendo los tiempos de espera y mejorando el aprovechamiento del canal.

Por otro lado, Stop & Wait, si bien es más simple de implementar, presenta un rendimiento limitado debido a su naturaleza secuencial, lo que lo hace más sensible a la latencia y a la pérdida de paquetes.

El uso de herramientas como Mininet permitió validar el comportamiento del sistema en entornos controlados, simulando condiciones reales de red y evidenciando la importancia de los mecanismos de retransmisión y control de flujo.

En conclusión, se logró cumplir con los objetivos del trabajo, desarrollando una solución robusta que garantiza la transferencia confiable de archivos sobre un protocolo no confiable como UDP, y permitiendo comparar el desempeño de distintos enfoques de recuperación de errores.